



AI and Machine Learning Project



Mekelle University Mekelle Institute of Technology (MIT)

Sentiment Analysis Project Documentation

Num	Name	ID Number
1	Temesgen Bahlbi	Mit/ur/135/12
2	Merhawi Gebreslassie	Mit/ur/103/12
3	Daniel Aregawi	Mit/ur/033/12
4	Mary Ephrem	Mit/ur/104/12
5	Berihu Esayas	Mit/ur/317/12
6	Goitom Gebregergs	Mit/ur/056/12
6	Hayle Tassew	Mit/ur/068/12
7	Selemun Gebrecherkos	Mit/ur/123/12

Table of Content

Introduction	1
Project Objectives	1
Methodology.....	1
Data Collection.....	1
Data Preprocessing	1
Model Selection	1
Training and Testing.....	1
Evaluation and Validation	1
Implementation	2
Libraries and Tools	2
Code Overview	2
Example Usage.....	3
Results and Analysis	3
Model Performance	3
Visualizations	4
Conclusion.....	6
Future Work.....	6
References	6

Introduction

This documentation provides a comprehensive overview of the Sentiment Analysis project, which aims to classify sentiments expressed in text data, specifically focusing on movie reviews and social media posts. The project leverages various machine learning models, including Naive Bayes, Support Vector Machines (SVM), Recurrent Neural Networks (RNN), and Convolutional Neural Networks (CNN), to analyze and predict sentiments.

Project Objectives

The primary objectives of the project are:

1. **Develop a Robust Sentiment Analysis Model:** Create a machine learning model capable of accurately classifying sentiments in text data as positive, negative, or neutral.
2. **Preprocess and Clean Data:** Ensure the dataset is clean and ready for analysis by removing noise and inconsistencies.
3. **Evaluate Multiple Machine Learning Algorithms:** Compare the performance of different algorithms to determine the most effective approach.
4. **Create a User-Friendly Interface:** Develop an intuitive interface for visualizing sentiment analysis results.
5. **Provide Comprehensive Documentation:** Offer detailed documentation and training materials for users to effectively utilize the sentiment analysis tool.

Methodology

Data Collection

The project utilizes two main datasets:

- **IMDB Movie Reviews:** Includes 50,000 movie reviews labeled as positive or negative

Data Preprocessing

The preprocessing steps include:

- **Cleaning:** Removing irrelevant characters, symbols, and stop words.
- **Normalization:** Converting text to a uniform format.
- **Tokenization:** Splitting text into individual words or tokens.
- **Lemmatization and Stemming:** Reducing words to their base or root forms.

Model Selection

The project evaluates several machine learning models:

- **Naive Bayes:** A probabilistic classifier based on Bayes' theorem.
- **Support Vector Machines (SVM):** A powerful classifier that finds the optimal hyperplane separating different classes.
- **Recurrent Neural Networks (RNN):** Deep learning models that capture sequential dependencies in text data.
- **Convolutional Neural Networks (CNN):** Deep learning models that capture local patterns in text data.

Training and Testing

The models are trained and tested using the preprocessed data. The training process involves:

- **Splitting the Dataset:** Dividing the dataset into training and testing subsets.
- **Feature Extraction:** Transforming text data into numerical representations.
- **Model Training:** Feeding the training data into the models and optimizing their parameters.

Evaluation and Validation

The models are evaluated using performance metrics such as accuracy, precision, recall, and F1 score. Cross-validation techniques are employed to ensure the models' generalizability.

Implementation

Libraries and Tools

The project uses the following libraries and tools:

- **pandas, numpy:** For data manipulation and numerical operations.
- **nlTK:** For natural language processing tasks.
- **scikit-learn:** For machine learning algorithms and evaluation metrics.
- **tensorflow, keras:** For deep learning frameworks.
- **matplotlib:** For plotting and visualization.

Code Overview

The implementation involves the following steps:

1. **Data Loading and Preprocessing:**
 - Load the IMDB dataset.
 - Check for and handle missing values.
 - Preprocess the text data using tokenization, stemming, and stopword removal.
2. **Feature Extraction:**
 - Convert text data into TF-IDF feature vectors.
3. **Model Training:**
 - Train Naive Bayes and SVM models using the TF-IDF features.
 - Train RNN and CNN models using padded sequences of tokenized text.
4. **Model Evaluation:**
 - Evaluate the performance of each model using accuracy, precision, recall, and F1 score.
 - Plot the training and validation accuracy for RNN and CNN models.
5. **Sentiment Prediction:**
 - Implement a function to predict the sentiment of new reviews using the trained models.

Example Usage

```
# Example usage of the predict_sentiment function
new_review = ""ved Finding Nemo! It's such a heartwarming story about family and friendship,
with beautiful animation and fun characters. Perfect for all ages!""
print("Sentiment (Naive Bayes):", predict_sentiment(new_review, nb_model, vectorizer))
print("Sentiment (SVM):", predict_sentiment(new_review, svm_model, vectorizer))
print("Sentiment (RNN):", predict_sentiment(new_review, 'rnn'))
print("Sentiment (CNN):", predict_sentiment(new_review, 'cnn'))
```

```
✓ Sentiment (Naive Bayes): Positive
Sentiment (SVM): Positive
1/1 ██████████ 0s 76ms/step
Sentiment (RNN): Positive
1/1 ██████████ 0s 43ms/step
Sentiment (CNN): Positive
```

Results and Analysis

Model Performance

- **Naive Bayes:** Achieved an accuracy of 0.85, with precision and recall values of 0.85 for both positive and negative sentiments.
- **SVM:** Achieved an accuracy of 0.89, with precision and recall values of 0.90 and 0.87 for negative sentiments, and 0.88 and 0.90 for positive sentiments.
- **RNN:** Achieved an accuracy of 0.51, indicating poor performance.
- **CNN:** Achieved an accuracy of 0.87, demonstrating strong performance.

Model Performance of Naive Bayes and SVM

Naive Bayes Performance:

	precision	recall	f1-score	support
negative	0.85	0.84	0.85	4961
positive	0.85	0.86	0.85	5039
accuracy			0.85	10000
macro avg	0.85	0.85	0.85	10000
weighted avg	0.85	0.85	0.85	10000

SVM Performance:

	precision	recall	f1-score	support
negative	0.90	0.87	0.88	4961
positive	0.88	0.90	0.89	5039
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

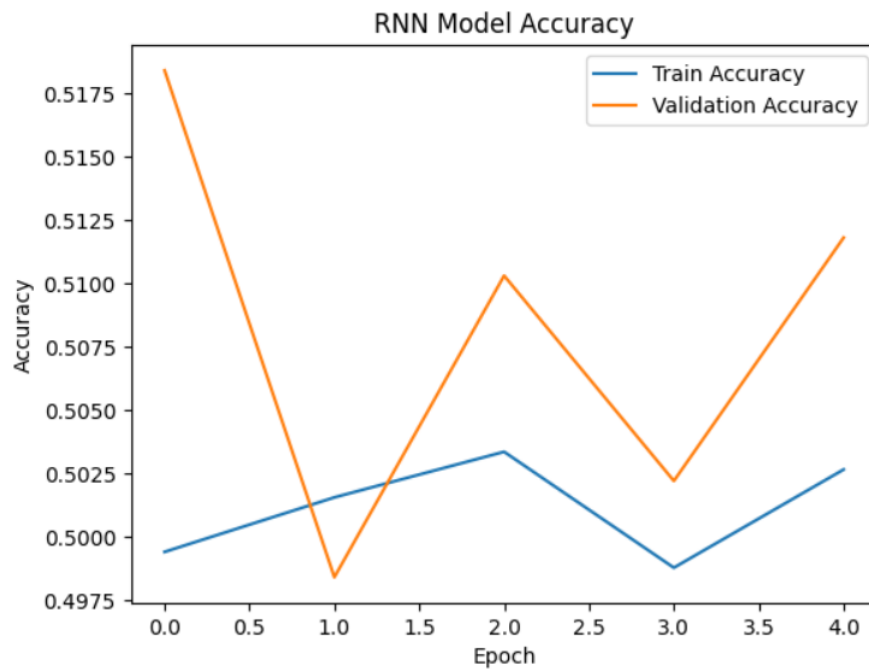
Model Performance of RNN and CNN

313/313  **23s** 75ms/step - accuracy: 0.5042 - loss: 0.6935
RNN Model Accuracy: 0.5117999911308289
313/313  **20s** 65ms/step - accuracy: 0.8709 - loss: 0.4243
CNN Model Accuracy: 0.8669000267982483

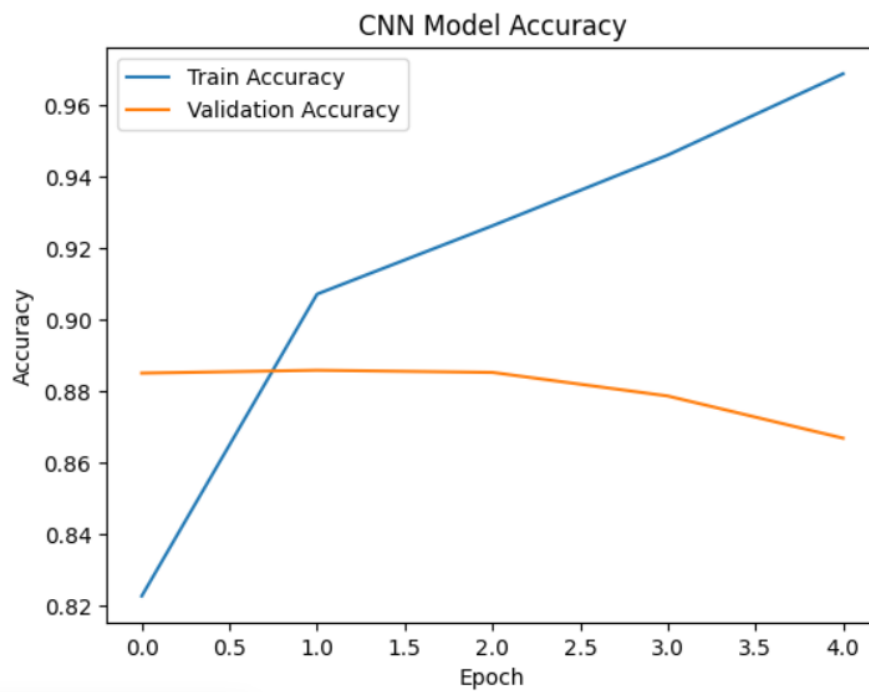
Visualizations

- **RNN Model Accuracy:** The training and validation accuracy plots show that the RNN model struggled to learn effectively, with accuracy hovering around 0.50.
- **CNN Model Accuracy:** The training and validation accuracy plots indicate that the CNN model performed well, with accuracy reaching up to 0.87.

Plot regarding to RNN Model Accuracy



Plot regarding to CNN Model Accuracy



Conclusion

The Sentiment Analysis project successfully developed and evaluated multiple machine learning models for classifying sentiments in text data. The CNN model demonstrated the best performance, making it a suitable choice for sentiment analysis tasks. The project also highlighted the importance of data preprocessing and model selection in achieving accurate and reliable results.

Future Work

- **Improve RNN Performance:** Investigate and address the issues causing the poor performance of the RNN model.
- **Expand Dataset:** Include more diverse datasets to improve the model's generalizability.
- **Enhance User Interface:** Develop a more sophisticated and interactive user interface for visualizing sentiment analysis results.
- **Explore Advanced Models:** Experiment with more advanced deep learning models, such as transformers, to further improve accuracy.

References

1. **IMDB Movie Reviews Dataset.** A dataset with 50,000 labeled movie reviews, used for training and evaluating sentiment analysis models. Retrieved from <https://ai.stanford.edu/~amaas/data/sentiment/>
2. **Scikit-learn Documentation.** Official documentation for Scikit-learn, providing machine learning algorithms and evaluation metrics. Retrieved from <https://scikit-learn.org/stable/>
3. **TensorFlow Documentation.** Official TensorFlow API reference for deep learning model development. Retrieved from https://www.tensorflow.org/api_docs
4. **Keras Documentation.** Official Keras documentation for building and training deep learning models. Retrieved from <https://keras.io/api/>